Terméknyilvántartó webtechnológiai projekt jegyzőkönyv

(Valkó Attila László, U7W4RZ)

Projekt célja

A projekt célja egy olyan webalkalmazás fejlesztése volt, amely lehetővé teszi a felhasználók számára a regisztrációt és bejelentkezést követően termékek kezelését.
A rendszer alapvető CRUD műveleteket biztosít (Create, Read, Update, Delete), így a felhasználók termékeket listázhatnak, létrehozhatnak, módosíthatnak és törölhetnek.

Regisztráció megvalósítása

A regisztráció Angular keretrendszerben, reaktív űrlap (Reactive Forms) használatával került megvalósításra. Az űrlap célja új felhasználó létrehozása validált adatokkal.

Űrlap felépítése

A felhasználó az alábbi adatokat adhatja meg:

- név (name)
- felhasználónév (username)
- jelszó (password)

Az űrlap Angular Material komponensekkel készült (`mat-form-field`, `mat-input`, `mat-card`), amelyek biztosítják az egységes és modern megjelenést.

Validáció

Az adatok ellenőrzése kliensoldalon történik a `Validators` segítségével:

Név:

- kötelező mező (`required`)
- minimum 3 karakter (`minLength(3)`)
- csak betűk és szóközök (`pattern: ^[a-zA-Z ]*$`)

Felhasználónév:

- kötelező
- minimum 3 karakter
- csak betűk és számok (`^[a-zA-Z0-9]+$`)

Jelszó:

- kötelező
- minimum 6 karakter
- csak betűk és számok

Hibák esetén a felhasználó azonnali visszajelzést kap `mat-error` elemek segítségével.

Hibakezelés megjelenítése

Az űrlap kétféle hibát kezel:

1. **Validációs hibák** – mező szinten jelennek meg
2. **Szerver oldali hiba** – általános hibaüzenet:

```
this.submitError = 'Registration failed. Please try again.';
```

Űrlap működése

A regisztráció folyamata az `onSubmit()` metódusban történik:

1. Az űrlap állapotának ellenőrzése
2. Hibás esetben minden mező “touched” állapotba kerül
3. Érvényes adatok esetén API hívás történik

```
this.userService.createUser(this.registrationForm.getRawValue()).subscribe(
  () => {
    console.log('User successfully created!');
    this.registrationForm.reset();        You, 19 hours ago • feat: add product mana
    this.ngZone.run(() => this.router.navigateByUrl('/login'));
  },
  (error) => {
    console.log(error);
    this.submitError = 'Registration failed. Please try again.';
  }
);
}
```

Backend kommunikáció

A frontend a `UserService` segítségével küld HTTP POST kérést a backend felé, amely létrehozza a felhasználót az adatbázisban.

Sikeres válasz esetén:

- az űrlap törlődik (`reset()`)
- a felhasználó átirányításra kerül a bejelentkezési oldalra

Navigáció

A felhasználó manuálisan is visszaléphet a bejelentkezési oldalra:

```
<div class="bottom-link">        You, 19 hours a
    Already have an account?
    <a routerLink="/login">Back to login</a>
</div>
```

Bejelentkezés megvalósítása

A bejelentkezési funkció célja, hogy a már regisztrált felhasználók hitelesítés után hozzáférjenek a védett funkciókhoz (pl. termékkezelés).

A megvalósítás Angular keretrendszerben, reaktív űrlap (Reactive Forms) használatával történt.

Űrlap felépítése

A felhasználó az alábbi adatokat adja meg:

- felhasználónév (username)
- jelszó (password)

Az űrlap Angular Material komponensekkel készült (`mat-form-field`, `mat-input`, `mat-card`), biztosítva az egységes megjelenést.

Űrlap működése

A bejelentkezési logika az `onSubmit()` metódusban található.

A folyamat lépései:

1. A felhasználó által megadott adatok kiolvasása:

```
const currentUser = this.loginForm.value?.username ?? '';
const currentPassword = this.loginForm.value?.password ?? '';
```

- Korábbi hibaüzenet törlése
- Felhasználók lekérése a backendről:

```
this.userService.getUsers().subscribe({
  next: (data) => {
    this.users = data;
    for (const user of this.users) {
      if (currentUser === user.username && currentPassword === user.password) {
        this.userService.setCurrentUser(user);
        this.router.navigate(['/products']);
        return;
      }
    }
    if (errorElement) {
      errorElement.innerHTML = 'Wrong username or password!';
    }
  },
  error: () => {
    if (errorElement) {
      errorElement.innerHTML = 'Login failed. Please check that the backend is running.';
    }
  }
});
```

Sikeres bejelentkezés

Amennyiben a felhasználónév és jelszó egyezik:

- a felhasználó eltárolásra kerül kliens oldalon
- átirányítás történik a termékek oldalára

Hibakezelés

Kétféle hiba kezelése történik:

1. Hibás bejelentkezési adatok

Ha nincs egyező felhasználó

2. Backend hiba

Ha a szerver nem elérhető:

Navigáció

A felhasználó a bejelentkezési oldalról eljuthat a regisztrációs oldalra:

```
    <mat-card-content>
        <h3 class="title">Don't have an account yet?</h3>
        <button mat-raised-button routerLinkActive="active" class="reg" routerLink="/registration">Sign Up</button>
    </mat-card-content>
</mat-card>        You, 19 hours ago • feat: add product management components and ser…
```

Dashboard megvalósítása

A dashboard oldal célja, hogy a bejelentkezett felhasználó számára egy központi felületet biztosítson, ahol megjelennek a saját adatai, valamint elérheti a rendszer további funkcióit (pl. termékkezelés).

Felépítés

A felület Angular Material komponensek segítségével készült (`mat-card`, `mat-button`), és két fő részből áll:

1. **Navigációs rész**
    a. „Products” gomb → termékkezelő oldalra navigál
2. **Felhasználói adatok**
    a. név megjelenítése
    b. felhasználónév megjelenítése
    c. kijelentkezés gomb

Felhasználói adatok kezelése

A komponens inicializálásakor (`ngOnInit`) meghívásra kerül a `getUser()` metódus.

Működés:

1. Lekéri az aktuális felhasználót a service-ből:

```
export class DashboardComponent implements OnInit {
  user = new User();
  username!: string;
  name!: string;

  constructor(private userService: UserService, private router: Router) { }

  ngOnInit(): void {
    this.getUser();
  }
```

Adatok megjelenítése

A felhasználó adatai a HTML-ben interpolációval jelennek meg:

```
    <mat-card-content>
        <div class="content"><span class="type">Name: </span>{{this.name.split('"')[1]}}</div>
        <div class="content"><span class="type">Username: </span>{{this.username.split('"')[1]}}</div>
        <button mat-raised-button routerLinkActive="active" (click)="logout()" class="menuBtn">Logout</button>
    </mat-card-content>
</mat-card>      You, 19 hours ago • feat: add product management components and ser…
```

Kijelentkezés

A kijelentkezés a `logout()` metódusban történik:

1. Felhasználó törlése:
2. Navigáció a login oldalra:

```
  logout(): void {
    this.user = new User();
    this.userService.setCurrentUser(null);
    this.router.navigate(['/login']);
  }
}
```

Termék létrehozása (Create)

A termék létrehozása funkció lehetővé teszi, hogy a bejelentkezett felhasználó új terméket vigyen fel a rendszerbe. A megvalósítás Angular reaktív űrlapokkal és Angular Material komponensekkel történt.

Űrlap felépítése

A felhasználó az alábbi adatokat adhatja meg:

- termék neve (`name`)
- mennyiség (`quantity`)
- mértékegység (`typeOfQuantity`)

Az űrlap `FormBuilder` segítségével lett létrehozva:

```
mainForm() {
  this.createProductForm = this.formBuilder.group({
    name: ['', [Validators.required, Validators.minLength(2), Validators.pattern('^[a-zA-Z0-9 ]+$')]],
    quantity: ['', [Validators.required, Validators.min(0), Validators.pattern('^[0.0*-9.9*]+$')]],
    typeOfQuantity: ['', [Validators.required]],
  });
}
```

Validáció

A mezők ellenőrzése kliens oldalon történik:

Név:

- kötelező
- minimum 2 karakter
- csak betűk, számok és szóköz

Mennyiség:

- kötelező
- nem lehet negatív (`min(0)`)
- szám típusú adat

Mértékegység:

- kötelező mező (pl. kg, db, liter)

A hibák `mat-error` segítségével jelennek meg közvetlenül az input mezők alatt.

Űrlap működése

A mentési folyamat az `onSubmit()` metódusban történik:

1. Űrlap állapotának ellenőrzése
2. Hibás esetben mezők megjelölése (`markAllAsTouched()`)

3. Érvényes adatok esetén API hívás

```
this.productService.createProduct(this.createProductForm.getRawValue()).subscribe(
  () => {
    console.log('Product successfully created!');
    this.createProductForm.reset();
    this.ngZone.run(() => this.router.navigateByUrl('/products'));
  },      You, 20 hours ago • feat: add product management components and ser…
  (error) => {
    console.log(error);
    this.submitError = 'Product could not be created.';
  }
);
}
```

Backend kommunikáció

A frontend a `ProductService` segítségével küld kérést a backend felé, amely eltárolja az új terméket az adatbázisban.

Termékek listázása (Read)

A funkció célja a rendszerben tárolt termékek megjelenítése, valamint az alap műveletek (szerkesztés, törlés) elérése.

Működés

A komponens betöltésekor:

1. Jogosultság ellenőrzés

```
getUser(): boolean {
  if (this.userService.getCurrentUser() === null) {
    this.router.navigate(['/login']);
    return false;
  }

  return true;
}
```

Termékek lekérése:

```
this.productService.getProducts().subscribe({
  next: (data) => {
    this.products = data;
    this.cdr.detectChanges();
  },
  error: (error) => {
    console.log(error);
    this.products = [];
    this.loadError = 'Products could not be loaded.';
    this.cdr.detectChanges();
  }
});
```

Megjelenítés

- *ngFor segítségével listázza a termékeket
- minden elemnél:
  - név
  - mennyiség
  - mértékegység

Műveletek

Szerkesztés:

Törlés:

```
12    <mat-card class="cardList" *ngFor="let product of products; let i = index">
13        <mat-card-title class="title">{{ product.name }}</mat-card-title>
14        <mat-card-content>
15            <div class="content"><span class="type">Quantity: </span>{{ product.quantity }}</div>
16            <div class="content"><span class="type">Type of Quantity: </span>{{ product.typeOfQuantity }}</div>
17        </mat-card-content>
18        <button mat-raised-button class="editBtn" (click)="edit(i)">Edit</button>
19        <button mat-raised-button class="deleteBtn" (click)="delete(i)">Delete</button>
20    </mat-card>
21
22    <div *ngIf="!loadError && products.length === 0">No products found.</div>
23 </div>
```

Termék módosítása (Update)

A funkció lehetővé teszi meglévő termékek adatainak szerkesztését.

Működés

1. Az URL-ből kiolvassa a termék azonosítóját:

```
ngOnInit(): void {
  if (!this.getUser()) {
    return;
  }

  this.productId = this.activatedRoute.snapshot.paramMap.get('id') ?? '';
  if (!this.productId) {
    this.router.navigateByUrl('/products');
    return;
  }

  this.productService.getProduct(this.productId).subscribe((product) => {
    this.editProductForm.patchValue(product);
  });
}
```

Mentéskor az űrlap először ellenőrzésre kerül, hogy minden kötelező mező helyesen ki van-e töltve és megfelel-e a validációs szabályoknak. Ha az adatok nem érvényesek, a folyamat megszakad, és a hibák megjelennek a felhasználónak.

Ha az adatok megfelelőek, a rendszer elküldi a módosított termék adatait a backend felé a termék azonosítójával együtt. A backend frissíti az adatbázisban a megfelelő rekordot az új értékekkel.

Sikeres mentés esetén a felhasználó visszakerül a termékek listájára, ahol már a frissített adatok jelennek meg. Sikertelen művelet esetén hibaüzenet jelenik meg, jelezve, hogy a módosítás nem történt meg.